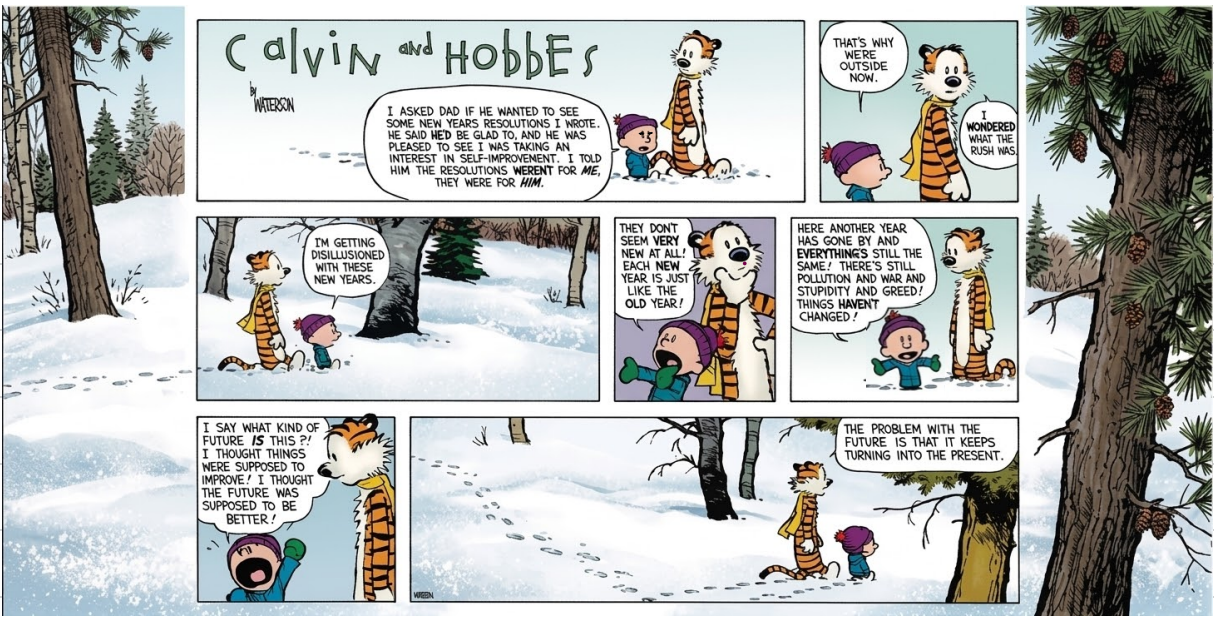
Enhance your Support Bot with RAG, Functions

We will begin 8:40 PM



Agenda:

1. RAG + LLM
2. RAG + LLM + INTENT ROUTER

Query Vector → Embed model → 1024 D

1 million PDF document → Customer Support

PDF1 → Embed model → 1024 D

PDF2 → Embed model → 1024 D

PDF3 → Embed model → 1024 D

⋮

PDF1M → Embed model → 1024 D

Query Vector → Embed model → 1024 D (q-v)

Cosine (q-v, Dc1)

Cosine (q-v, Dc2)

⋮

Dc1000 00)

1M — 10 Cluster

query Vector

10 + 100K

10 + 10 + 10K

$\frac{10 + 100K}{1M} = \frac{1}{10}$

K-1 — 100K

K2 — 100K

K3 — 100K

K1 — 100K

CK1 — 10K

CK2 — 10K

⋮

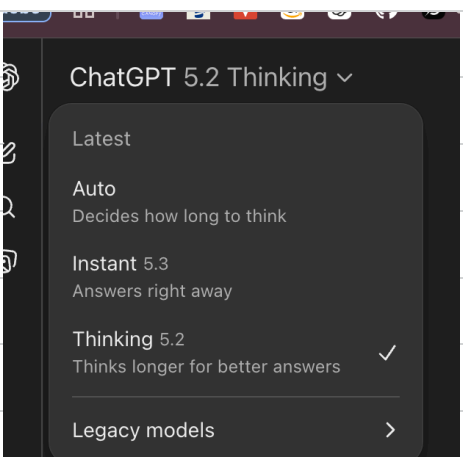
CK10 — 10K

⋮

K10 — 100K

Part - 1

1. Load / download data
2. Chunking data - , Embed - embedding gemma
3. Chroma - db - create a collection
   Folder / table
4. Chroma db - add all embedded chunks
5. Query → Embedding
5. Chromadb query - top K sim chunks
6. Retrieve text for those chunks → LLM



Question → Router Model
↓ Thinking
↓ Instant

LLM - Route
- Medical →
- Finance →
- Chemistry →
- Programming →

RAG + LLM

Query → Intent detector → Creative Scr. (t1)
→ Character (t2)
→ Novel (t3)

Intent → tool → Backend → Rag - QA.
↓
Result. + Query → LLM → Final Ans.

LLM 1

```
→ Tool Selection & Execution (by intent)
   ├→ quote_search → tool_quote_search() → chunks_cache scan
   ├→ character_context → tool_character_context() → retrieve()
   ├→ stats → tool_book_stats() → manifest lookup
   ├→ compare → parse_compare_names() + 2x tool_character_context()
   ├→ creative_scene → tool_rag_retrieve() → Chroma query
   └→ rag_qa → tool_rag_retrieve() → Chroma query
```

API + ↓ — LLM
↓
F.A

Q. Who is Ez. Bennet

↳ Intent router → character context

Ez. bennet

Character read

Final Answer — LL(1) — OUT + 0